UE442 - Circuits numériques et microcontrôleurs
TP2 : Système à processeur IP

L'objectif de ce TP est de se familiariser avec l'utilisation d'un FPGA et la conception d'un système de type microcontrôleur fondé sur l'implémentation d'un processeur IP Nios II d'Altera et de périphériques d'entrées/sorties.

Présentation de la maquette d'expérimentation et des logiciels :

Il est obligatoire de lire les supports de TPs fournis et de consulter les documents de références en PDF fournis par le constructeur Intel-ALTERA.

Liste des documentations techniques :

mnl_avalon_spec.pdf	Avalon Interface Specifications
DE1_Basic_Computer.pdf	Basic Computer System for the Altera DE1 Board
DE1_UserManual.pdf	DE1 Development and Education Board User Manual
n2cpu_nii5v1.pdf	Nios II Classic Processor Reference Guide
ug_embedded_ip.pdf	Embedded Peripherals IP User Guide
tt_nios2_hardware_tutorial.pdf	Nios II Hardware Development, Tutorial
quartusII_help-DE1.doc	Utilisation de Quartus et la carte FPGA DE1 d'ALTERA

À partir des documents de référence relatifs à la carte DE1, il vous est demandé d'étudier les informations techniques fournies afin de répondre aux questions suivantes :

1. Établir un tableau recensant les composants physiques externes au FPGA et interfacés avec celui-ci sur la carte DE1. L'étude portera principalement sur les mémoires, les interrupteurs et les LED, en précisant le nombre d'éléments et leurs caractéristiques (nombre et type).
2. Identifier les mémoires présentes sur la carte en précisant, pour chacune, le bus de données, le bus d'adresses, la capacité et le type de mémoire : RAM, ROM, mémoire statique ou dynamique.
3. Expliquer le rôle du bus Avalon.
4. Lire l'ensemble du sujet de TP et répondre aux questions qui ne nécessitent pas de manipulation expérimentale.

I. Utilisation du processeur Nios II

On souhaite à présent réaliser le même chenillard réalisé en TP1 mais à l'aide d'un processeur programmé en C et associé à des périphériques d'entrées/sorties. Pour cela, on utilisera le processeur IP Nios II, implanté dans le FPGA, puis interfacé avec les composants nécessaires au fonctionnement du système, principalement les LED rouges et un timer.

Le processeur devra également accéder à une mémoire externe au FPGA afin d'y stocker le programme à exécuter ainsi que les données utilisées par les variables du programme C.

La mise en œuvre du système se déroulera selon les étapes suivantes :

1. À partir du modèle de conception Quartus fourni, utiliser l'outil Qsys pour analyser puis générer l'architecture du processeur et des périphériques nécessaires. Qsys produit une description VHDL de l'ensemble processeur + périphériques, qui sera ensuite intégrée au projet Quartus. Quartus réalisera alors la synthèse de cette description afin de générer le fichier de configuration du FPGA, appelé bitstream.

2. Intégrer dans le projet Quartus la partie générée par Qsys. L'architecture obtenue est alors considérée comme un composant, dont les signaux d'entrées/sorties correspondent aux différents périphériques intégrés au système.
3. Ne pas modifier les noms des signaux de l'entité principale, car ceux-ci sont déjà associés aux broches précises du FPGA dans le fichier d'affectation des pins.
4. Une fois la compilation Quartus réussie, utiliser Altera Monitor Program pour préparer le système, charger la configuration du FPGA, fichier SOF, compiler le programme C, le transférer sur la carte, puis l'exécuter. Cet outil intègre également un débogueur permettant de placer des points d'arrêt, de consulter la mémoire et de suivre pas à pas l'exécution du programme.

La figure 1 ci-dessous illustre un exemple d'architecture implantée sur le FPGA et différents périphériques intégrés sur la carte DE1.

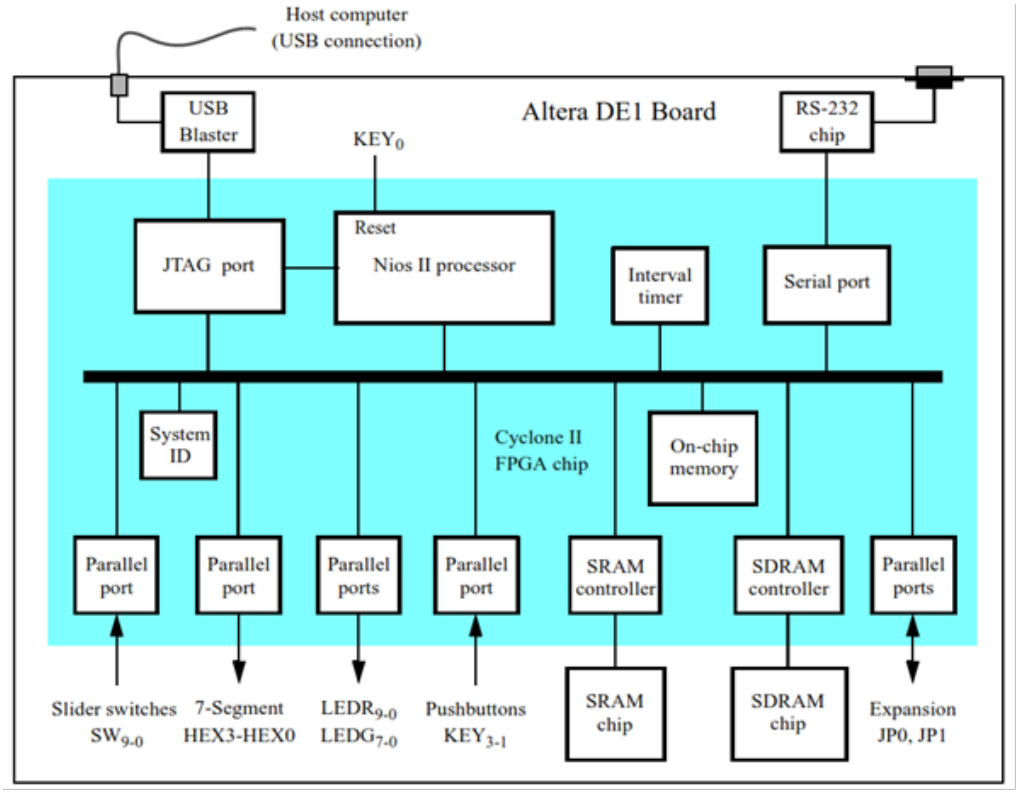
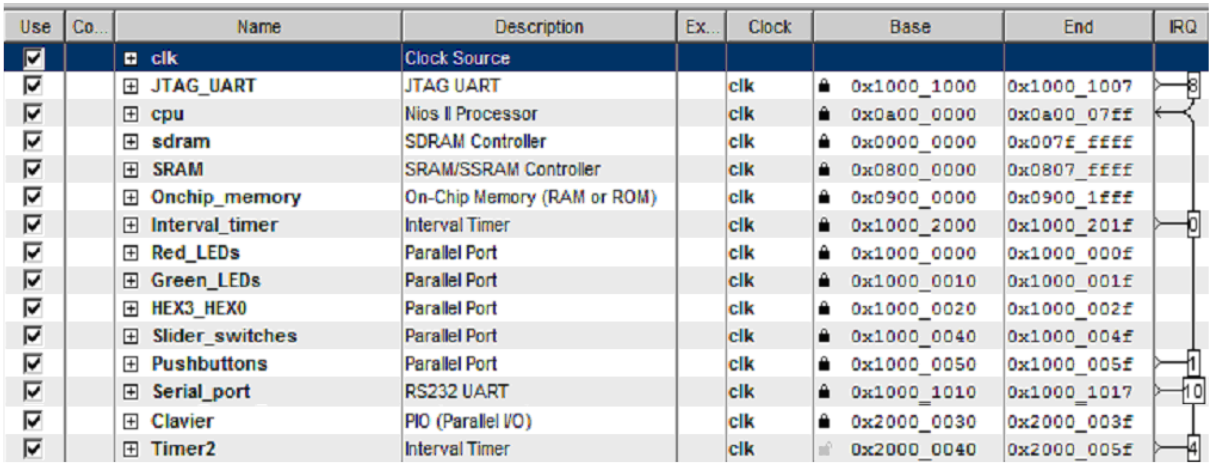


Figure 1

La figure 2 ci-dessous donne un exemple d'implantation (sur l'outil QSYS) d'une architecture de processeur NIOS-II avec ses interfaces périphériques qui seront reliés et utilisé par ce processeur.



Use	Co...	Name	Description	Ex...	Clock	Base	End	IRQ
☑		clk	Clock Source					
☑		JTAG_UART	JTAG UART		clk	0x1000_1000	0x1000_1007	8
☑		cpu	Nios II Processor		clk	0x0a00_0000	0x0a00_07ff	
☑		sdram	SDRAM Controller		clk	0x0000_0000	0x007f_ffff	
☑		SRAM	SRAM/SSRAM Controller		clk	0x0800_0000	0x0807_ffff	
☑		Onchip_memory	On-Chip Memory (RAM or ROM)		clk	0x0900_0000	0x0900_1fff	
☑		Interval_timer	Interval Timer		clk	0x1000_2000	0x1000_201f	0
☑		Red_LEDs	Parallel Port		clk	0x1000_0000	0x1000_000f	
☑		Green_LEDs	Parallel Port		clk	0x1000_0010	0x1000_001f	
☑		HEX3_HEX0	Parallel Port		clk	0x1000_0020	0x1000_002f	
☑		Slider_switches	Parallel Port		clk	0x1000_0040	0x1000_004f	
☑		Pushbuttons	Parallel Port		clk	0x1000_0050	0x1000_005f	1
☑		Serial_port	RS232 UART		clk	0x1000_1010	0x1000_1017	10
☑		Clavier	PIO (Parallel I/O)		clk	0x2000_0030	0x2000_003f	
☑		Timer2	Interval Timer		clk	0x2000_0040	0x2000_005f	4

Figure 2

1. Par comparaison entre les 2 figures ci-dessus, identifiez les périphériques de la figure 1 qui sont véritablement implantés dans l'environnement de l'outil QSYS.

Nous abordons maintenant la partie écriture du code C qui sera compilé puis exécuté sur le processeur NIOS. Quelques règles importantes à respecter et suivre :

- Le compilateur C a tendance à vouloir optimiser la génération de code et surtout l'accès aux variables. De ce fait, un accès à l'espace d'adressage pour lire une donnée pourrait être optimisée par le compilateur. Conséquences, les accès suivants ne vont pas lire le périphérique mais se contente de réutiliser la donnée déjà lue. Pour éviter cela, toujours déclarer les adresses et pointeur vers des périphériques avec le mot clé « volatile ».
- Tout accès à un périphérique nécessite de connaitre son adresse et de l'adresser avec un pointeur. Attention, l'architecture du NIOS dispose d'un cache mémoire data. Ce cache emmagasine les accès déjà effectués, conséquences, au premier accès la donnée est dans le cache puis les accès suivants réutilisent cette donnée. Cela implique par exemple, l'écriture vers des LED ne changera qu'une fois. Les écritures suivantes seront maintenues dans le cache. Pour désactiver ce cache pendant l'accès au périphérique, il suffit mettre à 1 le bit MSB (bit 31) de l'adresse du périphérique. Par exemple, adresse périphérique 0x00010020, deviendra 0x80010020.
- Attention aux types des périphériques, par exemple si le périphérique a 12 bits d'accès, il sera nécessaire de prévoir un short int pour pouvoir lire ou écrire l'ensemble des bits.
- L'accès à une donnée par pointeur prend en compte le type de la donnée. Exemple si p est un pointeur déclaré (volatile int * p). L'accès à la donnée pointée par *p implique un accès 32 bits. L'accès à la donnée *(p+1) se fera sur les 32 bits suivants.

2. À partir des explications fournies pendant la séance, mettre en œuvre un chenillard programmé en C. La table LUT contenant les motifs d'affichage sera définie dans le programme sous la forme d'un tableau en C. Une temporisation logicielle non bloquante sera utilisée afin de respecter une durée d'affichage d'environ 250 ms par motif.

L'extrait de programme ci-dessous illustre le principe de la fonction logicielle tempo ainsi que son interaction avec la partie chargée de l'affichage.

```
// fonction logicielle de temporisation
char tempo( ) {
        static  unsigned int  latence=0;
        if(latence++ > MAX_LATENCE) {
                latence =0;
                return 1;
        }
        return 0;
}
……
void main () {
……..
   while(1) {
        if (tempo()==1 ) {
                // réaliser l'affichage sur les LEDs
        }
}
```

3. Pourquoi la variable latence est elle déclarée avec le mot clé static ?

II. Utilisation d'un Timer

Pour mieux évaluer ce temps, des fonctions C seront données (voir annexe 2). On pourra mesurer le temps écoulé grâce au TIMER interfacé au bus du processeur NIOS-II. Ce TIMER précis a une période de 20ns (correspondant à la fréquence de 50MHz).

1. Mettre en œuvre une solution pour mesurer le temps de la boucle principale du programme, pour pouvoir estimer la valeur de la constante MAX_LATENCE au plus proche de 250ms. Pour information, il est possible d'utiliser la fonction **printf** qui permet d'afficher du texte sur la fenêtre console de l'outil MONITOR Program. Des fonctions d'utilisation du TIMER seront également mises à votre disposition.

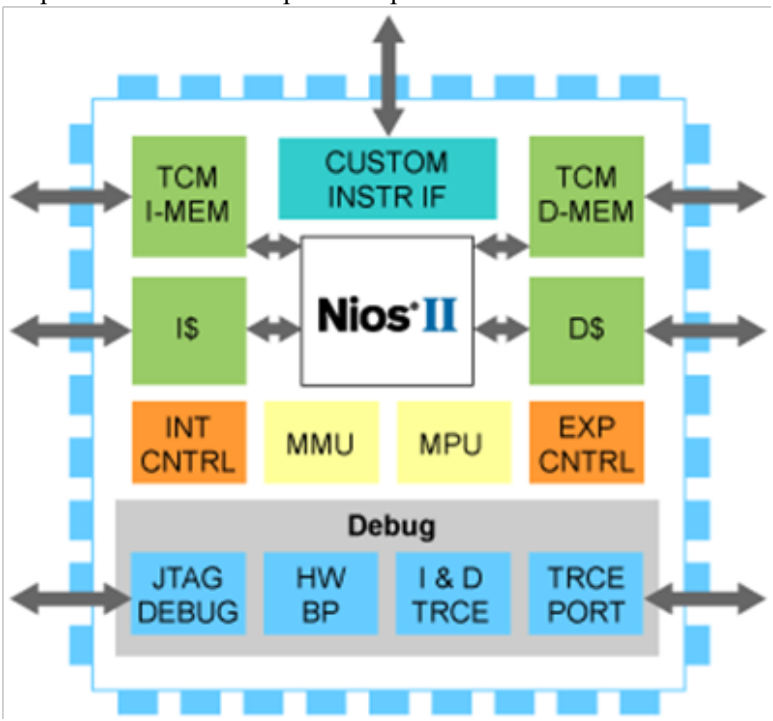
2. On souhaite améliorer le programme Chenillard en utilisant l'un des interrupteurs pour contrôler le sens de rotation des motifs. Pour cela, il faudra interfacer les interrupteurs (sous forme d'un port d'entrées/sorties, PIO) pour pouvoir les lires au niveau du processeur NIOS–II. On utilisera un bit de ces interrupteurs pour contrôler le sens de rotation de l'affichage de la table LUT des motifs.

ANNEXE 1

Les processeurs au cœur logiciel (Soft Core) affichent la plus grande flexibilité, les meilleures performances et le plus faible coût. Les développeurs peuvent choisir un nombre infini de configurations systèmes afin de répondre à leurs objectifs de performances et de coûts sans avoir recours à un ASIC. La production de l'ASIC (circuit spécifique) peut se faire lorsqu'on est assuré que le prototype conçu est parfaitement fonctionnel.

Le processeur Nios II d'ALTERA est un processeur 32-bit embarqué conçu spécialement pour la famille des FPGA Altera. Nios II incorpore de nombreuses améliorations sur l'architecture d'origine Nios, le rendant plus adapté pour une large gamme d'applications d'électronique embarquées et systèmes de contrôles.

Nios II est comparable à *MicroBlaze*, un processeur softcore des **FPGA Xilinx**. Contrairement au Microblaze, *Synopsys DesignWare* offre une licence de Nios II pour les *standard-cell ASICs*. Grâce à cette licence, les concepteurs peuvent utiliser Nios pour une production de masse d'ASIC.



L'architecture Nios II est une architecture **RISC** (Reduced Instruction Set Computer) qui est mise en œuvre entièrement dans un FPGA. La nature embarquée de ce type de processeur permet au concepteur de spécifier et générer un processeur sur mesure pour des besoins d'applications spécifiques. Les concepteurs peuvent étendre les fonctionnalités de base en y ajoutant une unité de gestion de mémoire prédéfinie, définir des instructions personnalisées ou des périphériques personnalisés.

Lors de la conception du processeur, nous utiliserons l'outil de haut niveau QSYS. Cet outil permet une conception par assemblage de briques, sous forme d'un Lego.
Le concepteur a en charge de réaliser les liaisons entre les différentes briques au travers du bus d'échanges AVALON. Ce bus est standardisé par ALTERA pour ses architecteurs. La figure en page 3 illustre une architecture telle qu'elle est vu au niveau de l'outil QSYS.

ANNEXE 2

Fonction en C pour l'accès au périphérique Interval Timer. Cela permet de mesurer le temps de calcul.

#define INTERVAL_TIMER_BASE ?????????? // indiquer l'adresse du périphérique Interval TImer.

// description des fonctions pour la mesure du temps

unsigned int Timer_period;

```
/*************************************************************************/
//   capturer la période du TIMER
/*************************************************************************/
int getperiod() {
        int val= *(((volatile unsigned int*)INTERVAL_TIMER_BASE)+2)|
                        (*(((volatile unsigned int*)INTERVAL_TIMER_BASE)+3)<<16);
        Timer_period=val;
        return val;
}

/*************************************************************************/
//   démarrer le TIMER
/*************************************************************************/
void start_timer() {
        *(((volatile unsigned int*)INTERVAL_TIMER_BASE)+1)=0x06;
}

/*************************************************************************/
//   capturer la valeur instantanée du TIMER
/*************************************************************************/
unsigned int getsnap() {
        *(((volatile unsigned int*)INTERVAL_TIMER_BASE)+4)=0;
        unsigned int val= *((volatile unsigned int*)(INTERVAL_TIMER_BASE)+4)|
                        (*(((volatile unsigned int*)INTERVAL_TIMER_BASE)+5)<<16);
        return val;
}

/*************************************************************************/
//   Calcul la différence entre 2 instant du Timer
// valeur exprimé en Tick          1 tick = 20ns
/*************************************************************************/
// calcul la différence v2-v1 modulo periode du Timer
unsigned int getsnapdif(int v1,int v2){
    if(v1>=v2){
         return (v1-v2);
    }
    return (Timer_period-v2+v1);
}
```